

TEMA N° 4



TEMA 4 ITERACIÓN, REFINAMIENTO Y DEFENSA FINAL



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Identificar, aislar y corregir errores lógicos** en tus aplicaciones mediante técnicas profesionales de depuración y diseño de pruebas unitarias, garantizando que el software entregado a la comunidad de Pailón funcione de manera óptima y sin fallos imprevistos.
2. **Co-redactar y refinar con precisión académica** los capítulos de conclusiones, recomendaciones y anexos de tu documento de grado, utilizando herramientas de Inteligencia Artificial de forma ética, estructurada y adaptada a las exigencias normativas de nuestro CEA.
3. **Diseñar una presentación de diapositivas de alto impacto y dominar las competencias de oratoria**, control del tiempo y expresión corporal necesarias para defender con éxito tu



proyecto sociocomunitario productivo ante el tribunal evaluador, consolidando tu perfil como Técnico en Sistemas Informáticos.



PRÁCTICA



Imagine que ha pasado los últimos meses desarrollando un sistema web de gestión de inventarios y ventas para la Asociación de Comerciantes del **Barrio Central Fátima**, justo al lado de la iglesia principal de Pailón. El sistema parecía funcionar de maravilla en las computadoras del nuevo y moderno módulo educativo del **CEA Herman Ortiz Camargo** en el **Barrio Saó**. Los docentes revisaron las pantallas, las bases de datos estaban conectadas y la interfaz gráfica lucía impecable.

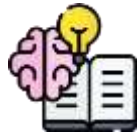
Faltando solo dos semanas para la defensa final del proyecto de grado, el equipo de estudiantes decide realizar una prueba piloto en vivo en tres de los comercios más grandes del Barrio Central Fátima. El día de la prueba, con los comerciantes observando entusiasmados, el sistema permite registrar los productos individualmente sin inconvenientes. Sin embargo, en el momento en que un cliente intenta comprar un combo de herramientas que aplica un descuento especial por volumen, la pantalla se congela, el total de la factura se calcula en un valor negativo exorbitante y la base de datos se bloquea de forma indefinida. El software no muestra ningún mensaje de error en la pantalla, simplemente se comporta de una manera errática e incomprensible.

Los comerciantes miran con desconfianza, los estudiantes entran en pánico y el documento de Word de la tesis aún tiene vacíos críticos en el capítulo de recomendaciones y conclusiones. Por si fuera poco, el equipo todavía no ha estructurado las diapositivas de la exposición y ningún miembro del grupo ha practicado hablar en público frente a un público exigente.

¿Cómo es posible que un programa que no mostraba errores de sintaxis en el laboratorio del CEA falle de forma tan catastrófica ante un caso real en el entorno productivo de Pailón? ¿De qué manera pueden los estudiantes rastrear el origen exacto de este fallo en el código, corregirlo de forma definitiva, documentar la solución éticamente con apoyo de herramientas modernas y preparar una defensa oral impecable que convenza al tribunal más estricto? Este escenario real



nos introduce en la necesidad absoluta de iterar, refinar y defender con solidez científica nuestro trabajo de fin de estudios.



TEORÍA



4.1. DEPURACIÓN DE ERRORES LÓGICOS Y PRUEBAS UNITARIAS (DEBUGGING)

Para un Técnico en Sistemas Informáticos, la programación no concluye cuando el código compila o cuando la página web carga por primera vez. El proceso de desarrollo requiere una etapa rigurosa de control de calidad. Los errores en el software se dividen fundamentalmente en errores de sintaxis (que el compilador o intérprete detecta inmediatamente debido al mal uso de las reglas del lenguaje) y errores lógicos. Estos últimos son los más peligrosos: el programa se ejecuta sin lanzar alertas obvias, pero los resultados numéricos o los flujos de comportamiento son completamente incorrectos.

La depuración (*debugging*) es el proceso metódico de localizar, analizar y solucionar estos fallos ocultos. En lugar de adivinar qué línea está fallando o llenar el código de impresiones genéricas, el Técnico utiliza herramientas de depuración integradas en los entornos de desarrollo (como VS Code o PyCharm) para establecer "puntos de interrupción" (*breakpoints*). Un breakpoint detiene la ejecución del programa en una línea específica, permitiendo inspeccionar el valor exacto de las variables en la memoria en ese preciso instante y avanzar la ejecución paso a paso.





Para evitar que estos errores lógicos reaparezcan al realizar modificaciones futuras (*regresiones*), se implementan las pruebas unitarias (*unit testing*). Una prueba unitaria es un pequeño fragmento de código diseñado para verificar de forma automatizada que una función o componente específico del sistema cumpla exactamente con el comportamiento esperado bajo condiciones dadas.

A continuación, se presenta un ejemplo práctico en Python que simula el error de cálculo del descuento planteado en nuestro caso de estudio del Barrio Central Fátima, mostrando primero el código defectuoso, su corrección lógica y la correspondiente prueba unitaria automatizada.

Python

```
# Módulo de cálculo de facturación para los comerciantes de Pailón
# Archivo: facturacion.pydef calcular_total_con_descuento(precio_unitario,
cantidad, porcentaje_descuento):    # ERROR LÓGICO: Se está restando el
porcentaje directamente del subtotal monetario    # En lugar de restar el valor
del descuento, se altera la operación aritmética    subtotal = precio_unitario *
cantidad # Multiplica el precio por la cantidad real    total_incorrecto =
subtotal - porcentaje_descuento # Operación errónea: resta directa    return
total_incorrecto # Retorna un valor distorsionado o negativodef
calcular_total_corregido(precio_unitario, cantidad, porcentaje_descuento):    #
SOLUCIÓN: Calcular el monto exacto equivalente al porcentaje de descuento
subtotal = precio_unitario * cantidad # Obtiene el costo base de los productos
monto_descuento = subtotal * (porcentaje_descuento / 100.0) # Determina el valor
del descuento    total_correcto = subtotal - monto_descuento # Resta el
descuento real al subtotal    return total_correcto # Retorna el valor final
exacto y verificado
```

Para asegurar que la función de cálculo corregida nunca vuelva a fallar ante cambios imprevistos en el sistema, el Técnico escribe un script de pruebas unitarias utilizando la librería estándar unittest.

Python

```
# Script de pruebas unitarias automatizadas para el sistema de facturación
# Archivo: test_facturacion.pyimport unittest # Importa el framework estándar de
pruebas unitarias de Pythonfrom facturacion import calcular_total_corregido #
Trae la función que deseamos evaluarclass
TestFacturacionPailon(unittest.TestCase): # Hereda la clase base para crear
escenarios de prueba    def test_descuento_combo_herramientas(self): # Define el
caso de prueba para el Barrio Central Fátima    # Escenario: 2 combos a 150
BS cada uno, con un 10% de descuento aplicado    precio = 150.0 # Define el
```



```

precio_unitario_del_producto_en_bolivianos      cantidad = 2 # Establece la
cantidad_de_articulos_comprados                  descuento = 10 # Define el porcentaje de
descuento_otorgado                               resultado_esperado = 270.0 # El subtotal es
300, el 10% es 30, el total debe ser 270         resultado_real =
calcular_total_corregido(precio, cantidad, descuento) # Ejecuta la función del
sistema                                           # Compara el resultado real con el esperado;
si no coinciden, la prueba falla explícitamente
self.assertEqual(resultado_real, resultado_esperado, "
El cálculo del descuento presenta fallos lógicos.")
if __name__ == "__main__": # Verifica si el archivo se ejecuta de manera directa
unittest.main() # Inicia la ejecución automática de todas las pruebas declaradas
en la clase

```

4.2. REDACCIÓN TÉCNICA DE LOS CAPÍTULO FINALES DEL DOCUMENTO CON IA

El documento de grado es el respaldo científico y metodológico del software desarrollado. Los capítulos finales, específicamente las Conclusiones y las Recomendaciones, suelen presentar dificultades para los estudiantes debido al nivel de abstracción y formalismo académico requerido. En la actualidad, las herramientas de Inteligencia Artificial Generativa (como Gemini o ChatGPT) actúan como asistentes de redacción sumamente potentes, siempre y cuando el Técnico ejerza un rol de director y auditor crítico de los textos generados.

Las conclusiones deben responder directamente a los objetivos específicos planteados al inicio del proyecto. No son un resumen de actividades, sino la demostración del valor tecnológico alcanzado. Las recomendaciones, por su parte, trazan la hoja de ruta para futuras mejoras del sistema, adaptándose a las limitaciones y oportunidades reales descubiertas en el entorno de despliegue.

Para obtener textos de alta calidad académica de una IA, el secreto radica en el diseño del *prompt* (instrucción detallada). Un *prompt* profesional debe incluir rol, contexto local, datos de entrada precisos y restricciones de formato.

A continuación, se expone un ejemplo de *prompt* estructurado para la elaboración de las conclusiones de un proyecto de red de datos inalámbrica destinado a la infraestructura del nuevo módulo del **CEA Herman Ortiz Camargo**:

Prompt de Redacción Académica:



"Actúa como un redactor académico senior en ingeniería de sistemas y educación técnica. Ayúdame a redactar dos conclusiones formales para mi proyecto de grado titulado 'Implementación de una infraestructura de red local Wi-Fi segura para el nuevo módulo del CEA Herman Ortiz Camargo en el Barrio Saó, Pailón'.

Toma en cuenta los siguientes datos reales del proyecto:

1. Se instalaron 3 puntos de acceso distribuidos que cubren las aulas de informática y administración.
2. Las pruebas de carga demostraron soporte para 60 conexiones simultáneas de estudiantes sin degradación de velocidad.
3. Se configuró una VLAN independiente para la administración educativa y otra para los estudiantes, aislando el tráfico.

Restricciones de estilo: Utiliza un tono formal, técnico y analítico. Evita opiniones personales o palabras vacías. Estructura el texto en párrafos fluidos y limpios sin usar viñetas, listados ni numeraciones de ningún tipo."

El resultado generado por la IA bajo estas directrices proporciona una base sólida que el estudiante debe revisar, ajustar con sus propias palabras y validar con la realidad institucional, garantizando la autenticidad y el rigor del documento final.

4.3. CREACIÓN DE LA ESTRUCTURA DE LA PRESENTACIÓN EN DIAPOSITIVAS

La presentación visual no es un sustituto del documento escrito, sino un guion gráfico de soporte que debe sintetizar meses de trabajo técnico en un espacio de tiempo muy limitado (generalmente 20 minutos). Un error crítico y recurrente es sobrecargar las diapositivas con textos interminables que fatigan al tribunal y empujan al estudiante a limitarse a leer la pantalla.

El diseño visual debe responder a la regla de la simplicidad: fondos claros con alto contraste, tipografías legibles a distancia (como Arial o Helvetica, con tamaños mínimos de 24 puntos para cuerpo de texto) y una presencia mayoritaria de esquemas, diagramas de arquitectura de software o capturas de pantalla del sistema en funcionamiento.



A continuación, se detalla la estructura estándar y equilibrada para organizar las diapositivas de la defensa final de un proyecto de Systems Informáticos en el CEA:

Número de Diapositiva	Contenido Principal y Enfoque Didáctico	Tiempo Sugerido
Diapositiva 1	Carátula Oficial: Título del proyecto, nombres de los autores, logotipo del CEA e identificación del módulo formativo.	0.5 a 1 minuto
Diapositiva 2	Introducción y Problema: Diagnóstico de la necesidad tecnológica detectada en los barrios de Pailón (ej. falta de automatización).	2 minutos
Diapositiva 3	Objetivos del Proyecto: Exposición del objetivo general y los objetivos específicos que marcaron el rumbo técnico del trabajo.	1.5 minutos
Diapositiva 4	Justificación y Sustento Social: Explicación del impacto directo del software en el desarrollo productivo y comunitario local.	1 minuto



Diapositiva 5	Ingeniería del Proyecto - Arquitectura: Diagrama de bloques que muestre la conexión de bases de datos, servidores y frontend.	4 minutos
Diapositiva 6	Demostración Práctica (Software): Capturas de pantalla o un video breve mostrando los flujos principales y validaciones del código.	5 minutos
Diapositiva 7	Conclusiones y Logros: Respuestas concretas a los objetivos planteados y demostración de la viabilidad técnica conseguida.	3 minutos
Diapositiva 8	Recomendaciones y Cierre: Propuestas de mantenimiento, escalabilidad del sistema y agradecimiento formal al tribunal.	2 minutos

4.4. ENSAYO, ORATORIA Y MANEJO DEL TIEMPO DE EXPOSICIÓN

Tener un software extraordinario y un documento impecable no garantiza la aprobación si el Técnico no logra comunicar sus ideas con claridad, seguridad y elocuencia. La oratoria académica no requiere condiciones innatas teatrales, sino una preparación metódica basada en técnicas de comunicación efectiva y simulación repetitiva.

El manejo de la voz es el primer pilar de la exposición. El estudiante debe modular el volumen para asegurar que los miembros del tribunal, sentados al fondo de las salas del CEA, escuchen cada término técnico sin necesidad de gritar. El ritmo del habla debe ser pausado; la prisa transmite nerviosismo y confusión. Introducir silencios estratégicos de dos segundos justo antes de revelar un logro importante del proyecto capta de inmediato la atención del auditorio.

El lenguaje corporal complementa las palabras. La postura debe ser erguida, manteniendo los pies alineados con los hombros para proyectar estabilidad. Los brazos deben usarse para enfatizar los diagramas expuestos en la pantalla, evitando cruzar las manos atrás, guardarlas en los bolsillos o sujetar lapiceros que provoquen movimientos repetitivos distractores. El contacto



visual es crucial: el expositor debe mirar directamente a los ojos de los evaluadores de forma alternada, demostrando que domina por completo la materia y que no teme al escrutinio académico.

Finalmente, el control estricto del tiempo se logra únicamente a través del ensayo con cronómetro en mano. El equipo debe practicar al menos cinco veces la exposición completa en las aulas del Barrio Saó. Esto permite ajustar las transiciones entre compañeros, recortar explicaciones secundarias que consumen valiosos minutos y asegurar que la presentación concluya con precisión suiza dentro del límite reglamentario fijado por la institución.

4.5. DEFENSA FINAL DEL PROYECTO ANTE TRIBUNAL EVALUADOR

La defensa final representa la cúspide del proceso formativo del Técnico. Es el acto formal donde se somete el trabajo al juicio crítico de un tribunal evaluador, compuesto generalmente por docentes del área y especialistas externos. La actitud del estudiante durante este evento debe combinar un profundo respeto protocolar con una firme seguridad técnica fundamentada en las horas dedicadas al desarrollo del proyecto.

El protocolo exige puntualidad estricta, vestimenta formal adecuada para la solemnidad académica y una organización previa impecable de los recursos físicos y digitales (computadora portátil cargada, cables de video correctos, copias del documento impreso y respaldos del código en unidades USB).

La fase de preguntas y respuestas es el momento donde muchos estudiantes flaquean por malinterpretar las observaciones del tribunal como ataques personales. Cuando un evaluador cuestiona una decisión de diseño (por ejemplo, por qué se eligió una base de datos relacional en lugar de una no relacional para el sistema de los campos deportivos del **Barrio 24 de Septiembre**), el Técnico debe responder con argumentos de ingeniería:

1. **Escucha Activa:** Tomar notas de la pregunta completa sin interrumpir jamás al evaluador.
2. **Madurez Intelectual:** Si el tribunal señala un error real u omisión en el código, se debe aceptar la observación con profesionalismo ("Es una excelente acotación, estimable tribunal. Tomaremos esa optimización para la siguiente versión del sistema").



3. **Defensa Argumentada:** Si la decisión fue deliberada, explicar los motivos técnicos y económicos locales ("Elegimos esta base de datos porque el servidor local del CEA cuenta con recursos de hardware optimizados para este motor y reduce los costos de mantenimiento de la institución").

4.6. INTEGRACIÓN CONTINUA Y PRUEBAS AUTOMATIZADAS LOCALES PARA PROYECTOS DE GRADO

Este apartado avanzado profundiza en metodologías modernas de desarrollo de software aplicadas al entorno del proyecto final.

A nivel profesional, la iteración de un sistema de software no se realiza de forma manual y caótica. Cuando varios Técnicos trabajan en el mismo código para un proyecto sociocomunitario (como un sistema de reserva de turnos para el coliseo municipal en el **Barrio 27 de Mayo**), fusionar los cambios de cada integrante puede generar incompatibilidades destructivas. Aquí es donde entra en juego el concepto de Integración Continua (CI - *Continuous Integration*).

Para un proyecto de grado, la integración continua puede implementarse a pequeña escala a través de scripts de automatización locales o ganchos de Git (*Git Hooks*). Cada vez que un desarrollador finaliza una corrección o añade un nuevo módulo al repositorio del proyecto, un sistema automatizado ejecuta inmediatamente todas las pruebas unitarias programadas de fondo. Si alguna prueba falla, el sistema bloquea la fusión del código, obligando al equipo a solucionar el error lógico antes de avanzar.

Bash

```
#!/bin/bash
# Ejemplo de un script de automatización local para control de calidad
# Archivo: verificar_proyecto.shecho "
Iniciando auditoría automatizada del sistema..."
# Ejecuta el formateador de código para asegurar estándares profesionalespython3
-m black .
# Ejecuta de forma automática todas las pruebas unitarias del directoriopython3
-m unittest discover -s tests
# Almacena el código de estado devuelto por las pruebasTEST_STATUS=?if [
TEST_STATUS -eq 0 ];
then    echo "¡ÉXITO! Todas las pruebas unitarias pasaron correctamente. El
sistema está listo para la defensa."    exit 0else    echo "
```



```
ERROR: Las pruebas unitarias fallaron. Revise el archivo facturacion.py
inmediatamente."      exit 1fi
```

4.7. AUDITORÍA DE SEGURIDAD Y OPTIMIZACIÓN DE CÓDIGO MEDIANTE ASISTENTES DE IA

Este apartado avanzado analiza cómo utilizar herramientas de inteligencia artificial para elevar los estándares de seguridad y rendimiento del código de grado.

Un sistema de información desarrollado para la comunidad de Pailón debe ser rápido y, sobre todo, seguro. Un error común en los proyectos de grado de estudiantes principiantes es dejar vulnerabilidades críticas en el código, tales como la inyección de código SQL o la exposición de credenciales administrativas en texto plano dentro de los archivos fuentes. Los asistentes de IA pueden actuar como auditores expertos de código automatizados.

El Técnico puede suministrar bloques específicos de funciones a la IA solicitando una revisión de seguridad bajo los estándares globales OWASP (Open Web Application Security Project). La IA analizará la lógica del código, detectará fugas de memoria o consultas inseguras a la base de datos y propondrá refactorizaciones avanzadas que mejoren los tiempos de respuesta del servidor del CEA.



A modo de ilustración, observe cómo un prompt preciso puede guiar a una IA para auditar un



módulo de autenticación de usuarios diseñado para el sistema de control de parques infantiles en el **Barrio Residencial Norte**:

Prompt de Auditoría de Código:

"Analiza la siguiente función de inicio de sesión escrita en Python y Flask para nuestro proyecto del Barrio Residencial Norte. Identifica posibles vulnerabilidades de inyección SQL o fallos en el manejo de contraseñas. Entrega el código completamente refactorizado utilizando consultas parametrizadas y buenas prácticas de seguridad industrial."

Este enfoque eleva el proyecto de grado de un nivel puramente académico a un estándar de desarrollo profesional de software listo para producción.

4.8. SIMULACIÓN INTERACTIVA DE JUEGO DE ROLES CON MODELOS DE LENGUAJE PARA LA PREPARACIÓN DE PREGUNTAS

Este apartado avanzado aborda el uso de tecnologías de IA conversacional para simular las condiciones de evaluación del tribunal en Pailón.

Uno de los mayores temores de los estudiantes en el **CEA Herman Ortiz Camargo** es no saber qué va a preguntar el tribunal evaluador durante la defensa final. Aunque las preguntas específicas varían, los patrones de evaluación académica suelen centrarse en la viabilidad, la metodología, las decisiones de arquitectura de software y el impacto sociocomunitario del proyecto en los barrios de Pailón.

Los Modelos de Lenguaje Avanzados (LLMs) permiten configurar un entorno interactivo de simulación de juego de roles (*roleplay*). El estudiante puede entrenar temporalmente a la IA para que adopte la personalidad de un profesor de sistemas sumamente estricto, especializado en bases de datos y redes, con el fin de sostener una sesión interactiva de preguntas de práctica.

Un ejemplo de prompt interactivo avanzado para ejecutar esta estrategia es el siguiente:

Prompt para Simulación de Tribunal:

"Quiero que actúes como el presidente del tribunal evaluador de proyectos de grado en el área de Sistemas Informáticos del CEA Herman Ortiz Camargo en Pailón. Eres un profesional muy



riguroso, técnico y analítico. Mi proyecto consiste en un sistema de control de asistencia y reserva de canchas para los complejos deportivos del Barrio 24 de Septiembre utilizando código QR.

Tu tarea es hacerme una pregunta técnica difícil a la vez sobre mi proyecto. Espera a que yo te responda en el chat, analiza mi respuesta críticamente de forma académica, dime en qué fallé o qué argumentos me faltaron, y luego plantéame la siguiente pregunta del examen. Comienza saludando formalmente e iniciando la primera pregunta."

Practicar de forma constante con este nivel de interactividad reduce la ansiedad del expositor, amplía el vocabulario técnico del estudiante y prepara argumentos de defensa sólidos para afrontar el examen de grado real sin sorpresas.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mitos Comunes en la Graduación	Realidades Técnicas Demostradas
Mito 1: Si el sistema de software compila perfectamente en mi computadora sin mostrar errores de texto en la consola, significa que la lógica matemática interna es infalible y no fallará ante los usuarios en Pailón.	Realidad: La compilación solo certifica que no hay errores de sintaxis gramatical. Los errores lógicos y de desbordamiento de datos solo se detectan mediante pruebas de estrés en vivo y metodologías estrictas de depuración (<i>debugging</i>).
Mito 2: El tribunal evaluador del CEA solo califica las diapositivas bonitas y la forma de hablar del expositor, por lo que el funcionamiento interno del código y el rigor de la tesis escrita pasan a un segundo plano.	Realidad: El tribunal evalúa de forma integral. Una excelente oratoria capta la atención, pero si el documento carece de sustento metodológico o el software falla en la demostración en vivo, el proyecto será reprobado de inmediato.

❑ Glosario de Términos

1. **Debugging (Depuración):** Proceso sistemático ejecutado por el programador para inspeccionar el estado interno de la memoria de un sistema de software con el fin de aislar, reproducir y eliminar fallos lógicos ocultos.
2. **Prueba Unitaria (Unit Test):** Código de software automatizado e independiente diseñado para aislar una función pequeña del programa principal y validar de manera matemática que devuelva los valores correctos bajo condiciones preestablecidas.



3. **Refactorización (Refactoring):** Técnica de desarrollo que consiste en modificar la estructura interna del código fuente para mejorar su legibilidad, eficiencia y seguridad, sin alterar bajo ningún concepto su comportamiento externo operativo.
4. **Prompt Engineering (Ingeniería de Prompts):** Disciplina técnica que se ocupa del diseño, estructuración y refinamiento de las instrucciones de texto enviadas a un modelo de Inteligencia Artificial para obtener respuestas de máxima precisión y rigor académico.
5. **VLAN (Virtual Local Area Network):** Red de área local virtual que permite segmentar de forma lógica una infraestructura física de red de datos para aislar el tráfico de usuarios por razones de rendimiento y seguridad informática.

□ **Ponte a Prueba**

1. Cuando un Técnico en Sistemas Informáticos utiliza un *breakpoint* (punto de interrupción) en su entorno de desarrollo durante la fase de refinamiento del proyecto, ¿qué acción está realizando específicamente en el sistema?

1. A) Borrando permanentemente las líneas de código que contienen fallas de sintaxis.
2. B) Deteniendo la ejecución del programa en una línea específica para inspeccionar en tiempo real el valor de las variables en memoria.
3. C) Compilando automáticamente el código fuente para subirlo a un servidor en la nube de forma directa.

2. ¿Cuál es el enfoque principal que debe tener la fase de recomendaciones dentro del documento final de tesis de un proyecto sociocomunitario productivo en Pailón?

1. A) Narrar detalladamente todas las dificultades económicas que tuvo el equipo durante el año de estudio.
2. B) Hacer un listado genérico de conceptos teóricos extraídos de enciclopedias web de internet.
3. C) Proponer mejoras tecnológicas futuras, planes de mantenimiento y vías de escalabilidad del software basadas en la realidad del entorno local.



3. Si durante la sesión de preguntas y respuestas de la defensa final en el CEA un miembro del tribunal detecta y demuestra un error real en una validación menor de tu software, ¿cuál debe ser la postura del Técnico?

1. A) Interrumpir inmediatamente al evaluador argumentando que el error fue causado por una falla de la energía eléctrica de Pailón.
2. B) Aceptar la observación con madurez y profesionalismo académico, comprometiéndose a incorporar la corrección para la optimización del sistema.
3. C) Guardar silencio absoluto durante el resto de la sesión y negarse a responder las preguntas subsiguientes del tribunal.

Respuestas correctas para verificación de estudio: 1-B, 2-C, 3-B.



VALORACIÓN



El desarrollo de soluciones tecnológicas de software y hardware para las comunidades de la provincia Chiquitos, y específicamente para la población de Pailón, conlleva una profunda responsabilidad ética, social y medioambiental que el Técnico no puede ignorar bajo ninguna circunstancia. La tecnología nunca opera en un vacío aislado; cada línea de código escrita en las aulas del **Barrio Saó** tiene el potencial de transformar la vida cotidiana de las personas, para bien o para mal.

Desde una perspectiva ética, el manejo de datos en sistemas de información locales (como las aplicaciones de ventas en el **Barrio Central Fátima** o los registros de usuarios en el **Barrio María Belén** y **Barrio San Antonio**) exige el cumplimiento estricto de los principios de confidencialidad y privacidad. Diseñar sistemas sin validaciones de seguridad o almacenar contraseñas en texto plano desprotegido constituye una negligencia profesional grave. El Técnico debe velar por la seguridad de la información de los pequeños comerciantes y ciudadanos locales, impidiendo filtraciones que puedan afectar su economía o su integridad personal.

Socialmente, los proyectos finales del **CEA Herman Ortiz Camargo** deben nacer de una vocación de servicio sociocomunitario productivo. El éxito de un software no se mide por la complejidad



abstracta de sus algoritmos, sino por su capacidad real para solucionar problemas humanos cotidianos, facilitando el trabajo de los comerciantes, optimizando los servicios públicos de los barrios o abriendo nuevos canales de comunicación para las familias de áreas circundantes como **Jardines de Pailón, Oto Waver y Ovidio Barberi**.

Finalmente, el impacto medioambiental de la informática es un desafío crítico de nuestro siglo. La constante actualización de sistemas y la obsolescencia de componentes de hardware generan toneladas de basura electrónica (*e-waste*) que contaminan los suelos de nuestra región. Un Técnico responsable extiende la vida útil de la infraestructura tecnológica existente mediante la optimización de software de bajo consumo de recursos. Al refinar el código para que se ejecute de manera ligera en computadoras antiguas o de gama baja, evitamos forzar a las instituciones educativas o pequeños negocios de barrios como **San Expedito, Las Misiones y San José** a desechar equipos funcionales, promoviendo un modelo de desarrollo tecnológico sostenible, equitativo y respetuoso con la Madre Tierra.



PRODUCCIÓN



LABORATORIO PRÁCTICO: DEPURACIÓN, PRUEBAS AUTOMATIZADAS Y REFINAMIENTO DOCUMENTAL DE UN SISTEMA DE GESTIÓN COMUNITARIO

Objetivo del Laboratorio: Aplicar de forma práctica e integrada las técnicas de depuración de errores lógicos, diseño de pruebas unitarias unitarias en Python, optimización de código mediante directrices de IA y estructuración de la documentación final para un sistema de control de asistencia desplegado en los campos deportivos del **Barrio 24 de Septiembre** de Pailón.

Recursos Necesarios:

1. Una computadora del módulo informático del CEA con Python 3 instalado.
2. Editor de código fuente (Visual Studio Code recomendado).
3. Acceso a un entorno interactivo de IA (para el ejercicio de redacción y auditoría).



Paso 1: Replicación del Software con Error Lógico

Abra su editor de código, cree un directorio llamado proyecto_pailon y guarde el siguiente fragmento de código en un archivo denominado asistencia.py. Este código simula el control de horas de entrenamiento de los jóvenes deportistas en la estación de tren y campos deportivos de Pailón, pero contiene una falla lógica grave en el cómputo de penalizaciones por retrasos.

Python

```
# Sistema de Gestión Deportiva - Barrio 24 de Septiembre
# Archivo: asistencia.py
def calcular_horas_validas(horas_totales, minutos_retraso):
    # ERROR LÓGICO: EL estudiante resta Los minutos directamente de las horas
    # Esto genera inconsistencias matemáticas fatales (ej. 10 horas - 30 minutos = -20 horas)
    horas_netas_incorrectas = horas_totales - minutos_retraso
    return horas_netas_incorrectas
def calcular_horas_validas_corregido(horas_totales, minutos_retraso):
    # SOLUCIÓN: Convertir los minutos de retraso a su equivalente exacto en fracciones de hora
    fraccion_horas_retraso = minutos_retraso / 60.0
    # Restar la fracción de tiempo calculada al total de horas registradas
    horas_netas_correctas = horas_totales - fraccion_horas_retraso
    return horas_netas_correctas
```

Paso 2: Automatización de la Validación mediante Pruebas Unitarias

Dentro del mismo directorio, cree un archivo llamado test_asistencia.py. Escriba la estructura de pruebas utilizando el componente unittest para comprobar el comportamiento del sistema ante un caso típico de retraso de un deportista pailoneño.

Python

```
# Script de validación automatizada de asistencia
# Archivo: test_asistencia.py
import unittest
from asistencia import calcular_horas_validas_corregido
class TestGestionDeportivaPailon(unittest.TestCase):
    def test_computo_retraso_entrenamiento(self):
        # Escenario: Un joven entrena 10 horas en total a la semana, pero acumula 30 minutos de retraso
        horas_registradas = 10.0
        minutos_atraso = 30
        # EL resultado correcto debe ser 9.5 horas reales válidas de entrenamiento
        resultado_esperado = 9.5
        resultado_real = calcular_horas_validas_corregido(horas_registradas, minutos_atraso)
        # Verificación de igualdad estricta
        self.assertEqual(resultado_real, resultado_esperado, "El sistema de horas descuenta de forma errónea los minutos.")
if __name__ == "__main__":
    unittest.main()
```



Ejecute la prueba en la terminal del sistema con el comando:

```
Bash
```

```
python3 test_asistencia.py
```

Verifique que la consola muestre un mensaje de éxito (OK), validando que la iteración y el refinamiento del código funcionan perfectamente.

Paso 3: Ejercicio de Refinamiento Documental con Asistencia de IA

Acceda a la herramienta de Inteligencia Artificial disponible en el aula del CEA y digite el siguiente prompt adaptado, introduciendo los resultados de su laboratorio para generar la redacción técnica de las conclusiones de este módulo:

Consigna para el Estudiante:

"Redacta un párrafo formal y académico para el capítulo de Conclusiones de mi documento de grado. El párrafo debe explicar cómo la implementación de pruebas unitarias automatizadas garantizó la confiabilidad del sistema de asistencia en los campos deportivos del Barrio 24 de Septiembre en Pailón, eliminando los fallos de cálculo lógicos en el control del tiempo de los usuarios. Evita usar adjetivos informales o listas."

Copie el texto generado por la IA, analícelo críticamente en su cuaderno de apuntes y realice los ajustes necesarios para que se acople perfectamente al estilo de redacción de su documento de tesis final.

Paso 4: Esbozo de la Diapositiva Técnica para la Defensa Final

En una hoja de papel o en un documento de texto plano, estructure el contenido textual exacto que colocará en la "Diapositiva 5: Ingeniería del Proyecto" para defender este laboratorio ante el tribunal del CEA. Diseñe la diapositiva aplicando las reglas aprendidas en la teoría:

1. **Título de la diapositiva:** Arquitectura de Control de Calidad y Pruebas Unitarias.
2. **Elemento Visual:** Bloque esquemático manuscrito que conecte el archivo asistencia.py con el validador test_asistencia.py.



3. Texto de Apoyo (Máximo 3 líneas cortas):

1. Automatización de pruebas lógicas mediante el framework unittest de Python.
2. Garantía de precisión matemática en el descuento de tiempos de usuarios comunitarios.
3. Arquitectura escalable sin regresiones de software para futuras fases de despliegue.

Presente los archivos de código funcionando, el texto de las conclusiones refinadas y el esbozo de su diapositiva al profesor del módulo para registrar la aprobación satisfactoria de su Reto Final de graduación.

RECURSOS ADICIONALES

1. **Documentación Oficial de Python - Librería unittest:** Guía técnica fundamental y detallada, con ejemplos avanzados de código, para profundizar en el diseño de pruebas unitarias automatizadas y asegurar la calidad del software. *(Enlace de referencia de la comunidad: docs.python.org/3/library/unittest.html)*
2. **Manual de Redacción Científica y Académica:** Recurso educativo abierto que provee normas de estilo, pautas de estructuración de textos de grado, uso correcto de citas y lineamientos metodológicos para la redacción de conclusiones y recomendaciones formales. *(Enlace de referencia de la comunidad: mdn.org/es/redaccion-academica-cientifica)*
3. **Guía Práctica de Oratoria Académica y Presentaciones de Alto Impacto (GitHub Guides):** Repositorio de metodologías libres sobre técnicas avanzadas de expresión corporal, manejo de la voz en auditorios públicos, control del miedo escénico y diseño minimalista de diapositivas para defensas de proyectos de ingeniería de software. *(Enlace de referencia de la comunidad: github.com/guides/oratoria-presentaciones-defensa-sistemas)*